05/10/2026

Continuing on with the analytical thermalization implementation. Have finished implementing the analytical thermalization into it's own little object. It is able to produce the expected system occupation, as well as plot $A(\omega)$. The simulation object generates a thermalization object such that it can compare the analytical vs numeric thermalization on the plot (as in Figure 1, Figure 2)

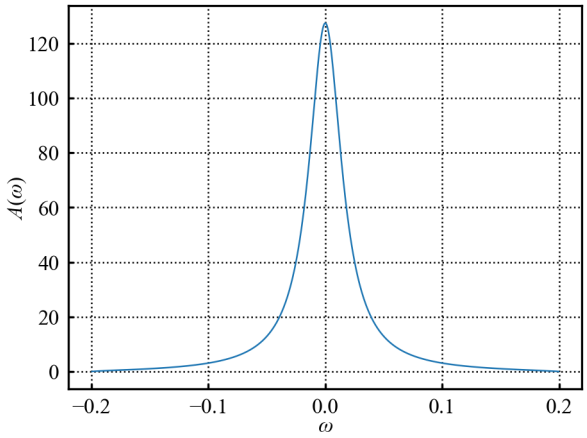


Figure 1: $A(\omega)$ for standard parameters

That took significantly longer than I would have liked; trying for symbolic automation of that integral was a poor idea.

Have also quickly setup the bones of the dual simulation system. Idk yet how to extract the dynamical map, but know that it will involve 2sims, 1 w/system starting occupied, the other w/unoccupied system. SO have setup a class that contains and runs and occupied and occupied simulation alongside basic utilities for it.

06/10/2026

Starting to look at the dynamical map; David has updated (Strachan 2026) with a quick outline.

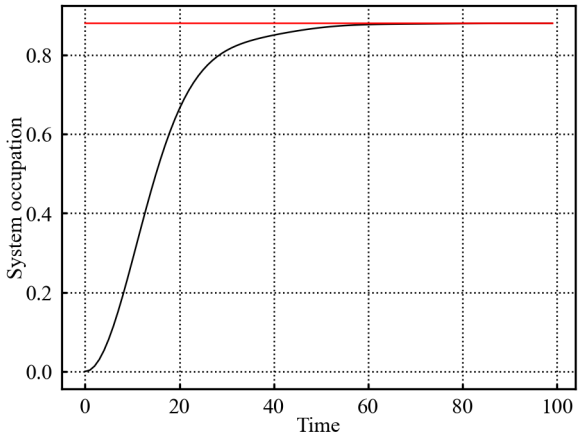


Figure 2: System occupancy over time with red line representing analytical thermalization value

Reading notes: Dynamical map extraction (Strachan 2026)

Because the simple model we're using only allows the system to be in a $|0\rangle$, or $|1\rangle$ state, we expect the dynamical map to be a 2x2 matrix of form:

$$\Lambda\hat{(}t) = \begin{bmatrix} \nu & \sigma \\ \alpha & \beta \end{bmatrix} \tag{1}$$

Considering it's action on some state of occupation p $|\psi\rangle = \begin{bmatrix} 1-p \\ p \end{bmatrix}$ and enforcing trace preservation yields:

$$\Lambda\hat{(}t)\,|\psi\rangle = \begin{bmatrix} \nu(1-p)+\sigma p \\ \alpha(1-p)+\beta p \end{bmatrix} \tag{2}$$

$$\downarrow$$

$$\nu(1-p)+\sigma p+\alpha(1-p)+\beta p = 1$$

$$(\nu+\alpha)+(\sigma+\beta-nu-\alpha)p = 1$$

which can only hold $\forall p$ if:

$$\hat{\Lambda(t)} = \begin{bmatrix} \nu & \sigma \\ 1-\nu & 1-\sigma \end{bmatrix} \tag{3}$$

Need to ensure "positivity" (unsure as to exactly why, but will roll w/it). Equation 2 maps initial occupation p to a new occupation $p^* = \nu(1-p) + \sigma p = \alpha(1-p) + \beta p$. Oh i see why positivity; p^* represents an occupation probability; negative values are nonsensical ergo $p^* \in [0,1]$ is a useful condition. Expressing $p^* = C_{11}(t)$ where C is the correlation matrix directly links the dynamical map s.t:

$$C_{11}(t) = \begin{cases} 1-\nu & C_{11}(0) = 0 = \textit{occupied} \\ \nu-\sigma & C_{11}(0) = 1 = \textit{unoccupied} \end{cases} \tag{4}$$

Therefore:

$$\hat{\Lambda(t)} = \begin{bmatrix} 1 - C_{occ}(t) & 1 - C_{occ}(t) - C_{unocc}(t) \\ C_{occ}(t) & C_{occ}(t) + C_{unocc}(t) \end{bmatrix} \tag{5}$$

where $C_{occ}(t) = C_{11}(t)$ for the initially occupied state, $C_{unocc}(t) = C_{11}(t)$ for the initially unoccupied state.

It is a touch unclear to me over which t $C_{11}(t)$ needs to be measured. My naive assumption is that it needs to be defined over all t of interest; so simulate till thermalization and then $\Lambda(t)$ encapsulates the dynamics over that time range. If so, the dynamical map would probably be stored as either a time-instantaneous dynamical map dictionary, or in an array. Far less storage required compared to the full correlation matrix (besides which the dynamical map is the primary object of analysis for later).

But assuming i'm correct, then implementing the dynamical map should simply be a matter of querying $C_{11}(t)$ for the occupied and unoccupied simulations respectively, and inputting those values into Equation 5.

Have added some empty functions and code comments to the dual sim object; nothing functional yet, just systems that will support this dynamical map extraction and use.

07/10/2026

Had supervisor meeting:

- Explained what i had done thus far
- Briefly discussed presentation:
  - Short explanation of what my project is and what the plan is
  - 1st slide, explain general research area
  - 2nd slide, explain specific problem and questions
  - explain general methods and background
  - show current results
  - outline plan
  - Note to self: is to a large degree pedagogical exercise; need to explain topic to unfamiliar peers; if they understand overview of what i'm doing, that's a win
- Note: can get system occupation from dynamical map; plotting this may be useful gut check to ensure everything is working
- Short-mid term plan:
  - Implement dynamical map into code
  - Implement memory kernel as first measure of non markovianity
  - Do more literature reading - I've spent a lot of time implementing a solid technical foundation for the project, but have spent comparatively little time delving into the literature and the theory
  - prep for presentation
- Provided additional papers; *(Cerrillo and Cao 2014) should outline the memory kernel stuff,
  - (Morillas-Rozas et al. 2026) may provide useful comparison of numerical kernel and analytical limit. This latter paper should provide insight into the subtleties and potential pitfalls if i don't treat this carefully

Consolidating the plan for the foreseable, i think it follows:

- Short term (pre presentation):
  - Wk of 05/10/2026: Implement dynamical map, implement plotting and analytical utilities for it in order to gut check it generally - maybe polish some of the plotting utilities to make them more flexible for later subplotting
  - Wk of 12/10/2026: Implement memory kernels and use it as case study to help guide literature deep dive
  - Wk of 19/10/2026: Block week out to work on presentation
  - Wk of 26/10/2026: Presentation slides are due this Wk - can take some time to practice presentation
  - Wk of 02/11/2026: Presentations occur
- Mid-long term (post presentation):
  - Finish off literature deep dive - this should give me a firm conceptual framework for the topic area and the different measures of non markovianity
  - Implement other memory definitions
  - Compare memory definitions on current setup
  - Figure out next steps w/insight gained at this point

09/10/2026

Have quickly implemented computing the dynamical map. Also implementing a "compute state from dynamical map" method which takes in an initial occupation probability, and evolves it using the dynamical map. Have also implemented a quick system plotting function which can plot this occupation over time. It doesn't match the basic simulation evolution; likely a typo somewhere in the maths.

hmmm, something odd is occuring.

Comparing a standard simulation Figure 3 to the dynamical map propagated occupation Figure 4 reveals some issues. The dynamical map evolution is clearly distinct in its scale ($n_{occ} \in [0, 8]$ vs $n_{occ} \in [0, 1]$ for dynamic and simulation derived occupations), as well as thermalizing slower. But further than that, the timestep seems to have an effect on the dynamical map derived occupation evolution.

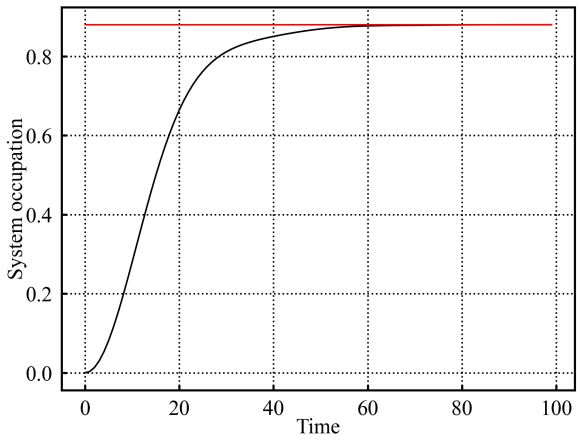


Figure 3: System occupation evolution computed from correlation evolution

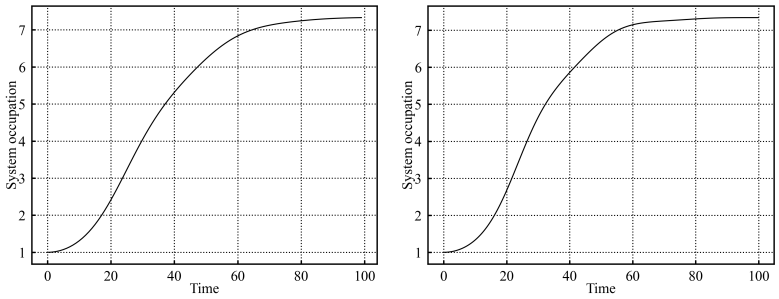


Figure 4: System evolution from occupied state propagated by dynamical map for $\Delta t = 1, 0.1$ respectively

Have quickly implemented unitary check for the dynamical map. Aaaand it isn't unitary; smth is wrong. Ok have removed one error; new evolution for dynamical map yields Figure 5.

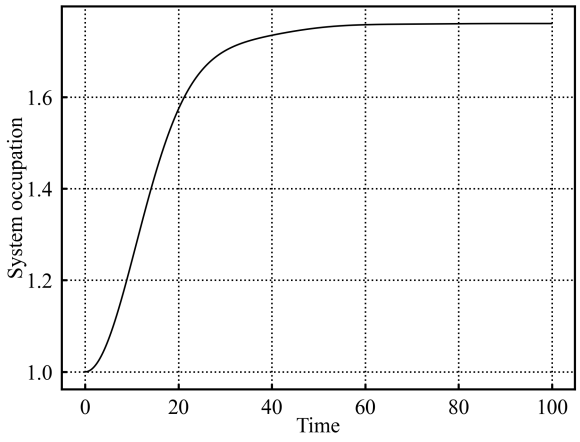


Figure 5: System occupation evolution computed from dynamical map after minor fix

This is still of the wrong scale, but is at least the right shape on first inspection. Quickly summarising what the code was doing:

$$|\psi(0)\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$
$$|\psi(t)\rangle = \Lambda(t) \cdot |\psi(t - \Delta t)\rangle$$

the fix switched it to:

$$|\psi(t)\rangle = \Lambda(t) \cdot |\psi(0)\rangle$$

The dynamical map at every steps applies to the initial state; not the previous state; which was causing issues. Its still not of the right magnitude which i suspect is due to the non unitary $\Lambda(t)$. It almost seems that a multiplication by 2 is occuring somewhere...

None of the dynamical maps are unitary for any time t.

Quickly checking some of the maths:

$$\Lambda\hat{(t)} \cdot \Lambda\hat{(t)}^{\dagger} = \begin{bmatrix} \nu & \sigma \\ 1-\nu & 1-\sigma \end{bmatrix} \cdot \begin{bmatrix} \nu^{\dagger} & (1-\nu)^{\dagger} \\ \sigma^{\dagger} & (1-\sigma)^{\dagger} \end{bmatrix} = \begin{bmatrix} \nu & \sigma \\ 1-\nu & 1-\sigma \end{bmatrix} \cdot \begin{bmatrix} \nu^{\dagger} & 1-\nu^{\dagger} \\ \sigma^{\dagger} & 1-\sigma^{\dagger} \end{bmatrix} = \begin{bmatrix} 2 & \nu+\sigma-2 \\ \nu^{\dagger}+\sigma^{\dagger}-2 & 2 \end{bmatrix} \tag{6}$$

If i haven't done my maths wrong; the expression Equation 3 isn't unitary as in Equation 6? I wouldn't bet against my doing the maths wrong; i am quite tired now and pen and papering the above… Which i think is a good sign to put this away for now. Tmr i can recheck the maths with better focus.

10/10/2026

Have quickly double checked the above maths Equation 6 using sympy:

```
>>> import sympy

>>> v, o = sympy.symbols('v o')
>>> lambdaMatrix = sympy.matrices.Matrix([[v,o],[1-
v,1-o]])
>>> conjugateMatrix = lambdaMatrix.H
>>> outputMatrix = sympy.simplify(lambdaMatrix * conjugateMatrix)
>>> print(sympy.latex(lambdaMatrix), "\cdot", sympy.latex(conjugateMatrix),"=", sympy.latex(ou
```

$$\begin{bmatrix} u & \sigma \\ 1-u & 1-\sigma \end{bmatrix} \cdot \begin{bmatrix} \overline{u} & 1-\overline{u} \\ \overline{\sigma} & 1-\overline{\sigma} \end{bmatrix} = \begin{bmatrix} \sigma\overline{\sigma}+u\overline{u} & -\sigma\overline{\sigma}+\sigma+u\overline{u}-u \\ -(\sigma-1)\,\overline{\sigma}-(u-1)\,\overline{u} & (\sigma-1)\,(\overline{\sigma}-1)+(u-1)\,(\overline{u}-1) \end{bmatrix} \tag{7}$$

Notation; when sympy exports to latex, it denotes conjugate using $\overline{x}$ instead of $x^{\dagger}$

Using sympy because its quite a flexible symbolic calculator, and I wanted to double check the pen and paper maths in Equation 6. Equation 7 seems to be entirely equivilant to Equation 6. If $\nu+\sigma=2$, $\nu,\sigma \in \mathcal{R}$, then $\Lambda\Lambda^{\dagger}=2I$ which is interesting because of the apparently $\times 2$ factor in Figure 5 compared to Figure 3. Given Equation 4, $\nu+\sigma=2$ imply $2C_{occ}+C_{unocc}=0$. No idea what this would imply; but it feels rather arbitrary…

Redoing some of the maths:

$$\hat{\Lambda(t)}\cdot|\psi(t=0)\rangle = \begin{bmatrix} \nu & \sigma \\ 1-\nu & 1-\sigma \end{bmatrix}\cdot\begin{bmatrix} 1-p \\ p \end{bmatrix} = \begin{bmatrix} \nu(1-p)+\sigma p \\ (1-\nu)(1-p)+(1-\sigma)p \end{bmatrix} = \begin{bmatrix} 1-p^* \\ p^* \end{bmatrix}$$

since $C_{11}(t) = n_{occ}$ it follows that $C_{11}(t) = p^*$ so:

$$(1-\nu)(1-p)+(1-\sigma)p = C_{11}(t) \tag{8}$$

Since $|\psi(p=0)\rangle = (1-p)\,|0\rangle + p\,|1\rangle = |0\rangle$, $p = 0$ implies an empty system, $p = 1$ an occupied system; so Equation 8 becomes:

$$C_{11}(t) = \begin{cases} 1-\sigma & C_{11}(0) = 0 = \textit{occupied} \\ 1-\nu & C_{11}(0) = 1 = \textit{unoccupied} \end{cases} \tag{9}$$

which differs to Equation 4 and would imply:

$$\hat{\Lambda(t)} = \begin{bmatrix} 1-C_{unocc}(t) & 1-C_{unocc}(t) \\ C_{unocc}(t) & C_{occ}(t) \end{bmatrix} \tag{10}$$

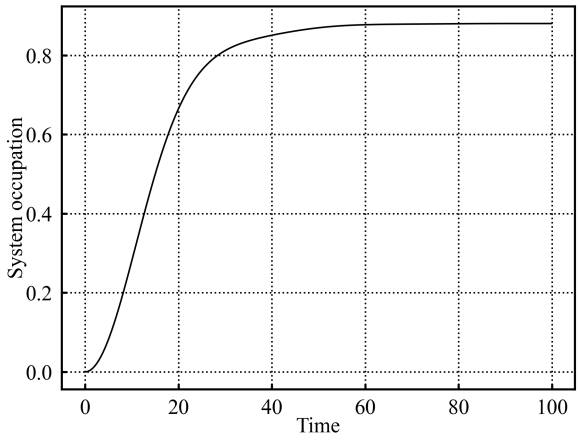


Figure 6: System occupation evolution computed from dynamical map Equation 10

Ok thats better; it seems the maths was misderived. Equation 10 still isn't unitary seemingly; but is reproducing the correct dynamics. Have tidied up some plotting utilities inc one which shows the system evolution from both occupied and unoccupied starting states:

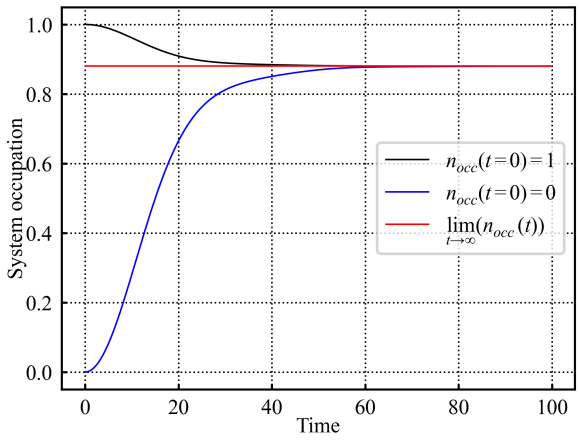


Figure 7: System occupation evolution computed using dynamical map for both occupied and unoccupied starts

The next steps require reading up on the different memory definitions. (Strachan 2026) notes memory kernels, information backflow and failure of complete positivity. Going through and implementing these one by one may be wise. This would help me familiarize myself w/the theory in one context, which should hopefully yield enough insight to make understanding the other definitions easier. As such, examining the memory kernels feels a decent place to start.

(Strachan 2026) notes memory kernels are defined in terms of the Nakajima-Zwanzig formalism. (Rivas and Huelga 2012) seems to have decent information on the topic, so will start there.

Reading notes: (Rivas and Huelga 2012)

Classical markovianity for context (p35) - stochastic processes w/Random variable $x(t)$ is markovian if the probability of x_n, t_n

is conditioned solely on the previous timestep s.t:

$$p(x_n, t_n|x_{n-1}, t_{-1}; ...; x_0, t_0) = p(x_n, t_n|x_{n-1}) \tag{11}$$

Taking a continuum limit yields:

$$p(x, t; x', t') = p(x, t|x', t')p(x', t')$$

Where x', t' take the place of x_{n-1}, t_{n-1} for some infintesimal step. Integrating over x' directly relates the two absolute probabilities:

$$p(x,t) = \int dx' p(x,t|x',t')p(x',t') = \int dx' K(x,t|x',t')p(x',t')$$

Which is of similar form to the summary in (Strachan 2026); with K being the memory kernel. This would make sense given that it literally represents the conditional probability of a given state x, t given some prior state(s) x', t'.

Sec 5.2 (p37) then outlines the specifics for quantum markovian evolution and its master eq. The key result of this section is the statement that QM markovian systems obey the Master equation:

$$\frac{d\rho(t)}{dt} = -i[H(t), \rho(t)] + \sum_k \gamma_k(t)(V_k(t)\rho(t)V_k^\dagger(t) - \frac{1}{2}\{V_k(t)V_k^\dagger(t), \rho(t)\}) \tag{12}$$

Where $H(t)$ is hermitian, $\gamma_k(t) > 0$. The section goes through the proof. Presumably $H(t)$ is the system hamiltonian, $\gamma_k(t)$ seemingly represent some eigenvalues; the meaning of V_k isn't clear. Would need to look at the proof in more detail; as well as try to parse the fairly abstract terms therein to get some intuition to what these terms represent. Not an immediate now thing.

Sec 6.1 (p52) introduces the Nakajima-Zwanzig Eq (albeit presumably in a markovian context). It sets up the hamiltonian:

$$H = H_a + H_b + V \tag{13}$$

Where H_a and H_b are the independent hamiltonians of the system and bath, whilst V represents the interaction between them (i wonder if this relates to the above V_k terms?). This results in a combined hilbert space $\mathcal{H} = \mathcal{H}_a \otimes \mathcal{H}_b$. It is apparently useful to define 2 orthogonal projection operators $\mathcal{P}, \mathcal{Q}$ s.t $\mathcal{P}^2 = \mathcal{P}$, $\mathcal{Q}^2 = \mathcal{Q}$, $\mathcal{P}\mathcal{Q} = \mathcal{Q}\mathcal{P} = 0$ defined as:

$$\mathcal{P}_\rho = Tr_b(\rho) \otimes \rho_b \tag{14}$$

$$\mathcal{Q}_\rho = (\mathcal{I} - \mathcal{P}_\rho)\rho \tag{15}$$

The method sets ρ_b to be the environments initial state as a convenience, and assumes initial thermal equilibrium s.t:

$$\rho_b = e^{-\beta H_b}[Tr(e^{-\beta H_b})]^{-1} \tag{16}$$

Inserting Equation 13 into the "Von Neuman Eq" yields:

$$\frac{d\rho(t)}{dt} = -i[H_a + H_b + V, \rho] \tag{17}$$

From context the Von Neuman Eq is $\frac{d\rho(t)}{dt} = -i[H, \rho]$

The method takes the projection operator on Equation 17. Note that this method uses the interaction picture; so seperates out the system/enviroment and interaction terms by defining:

$$\rho'(t) = e^{i(H_a+H_b)t}\rho(t)e^{-i(H_a+H_b)t}$$

For trace preservation reasons, the projector operators on Equation 17 can be written in this interaction picture as:

$$\frac{d\mathcal{P}\rho'(t)}{dt} = -i\mathcal{P}[V, \rho'] \tag{18}$$

$$\frac{d\mathcal{Q}\rho'(t)}{dt} = -i\mathcal{Q}[V, \rho'] \tag{19}$$

By redefining $\mathcal{V}(t) = -i[V, .]$, and taking several other steps, the generalized Nakajima-Zwanzig Eq results:

$$\frac{d}{dt}\mathcal{P}\rho'(t) = \mathcal{P}\mathcal{V}(t)\mathcal{P}\rho'(t) + \mathcal{P}\mathcal{V}(t)\mathcal{G}(t,0)\mathcal{Q}\rho'(0) + \int_0^t dt' \mathcal{P}\mathcal{V}(t)\mathcal{G}(t,t')\mathcal{Q}\mathcal{V}(t')\mathcal{P}\rho'(t') \tag{20}$$

$$\mathcal{G}(t,t') = \mathcal{T}e^{\int_{t'}^t ds \mathcal{Q}\mathcal{V}(s)}$$

Which is awful to look at. The paper notes 2 common assumptions; A) $\mathcal{P}\mathcal{V}(t)\mathcal{P} = 0$, B) assuming initial state $\rho(0) = \rho_a(0) \otimes \rho_b(0)$ is a product state of A, B. This simplifies Equation 20 to:

$$\frac{d}{dt}\mathcal{P}\rho'(t) = \int_0^t dt' \mathcal{K}(t,t')\mathcal{P}\rho'(t') \tag{21}$$

$$\mathcal{K}(t,t') = \mathcal{P}\mathcal{V}(t)\mathcal{G}(t,t')\mathcal{Q}\mathcal{V}(t') \tag{22}$$

With Equation 21 being similar in form to the summary given by (Strachan 2026). Some differences are present; $\mathcal{P}$ isn't present, and the interaction ρ' isn't acknowledged; i suspect that that may be simplified in this project, hence their absence. Further note that kernel homogeneity is a condition s.t $\mathcal{K}(t,t') = \mathcal{K}(t-t')$ (which is also the form given by (Strachan 2026)) which is apparently solvable by laplace eq.

The paper continues in sec 6.2 (p56) and sec 6.3 (p75) by examining a weak coupling limit and a singular coupling limit respectively. These are both long and involved sections; and i'm not sure how relevant they may be. That said however, there are examples in sec 6.2.5 (p67) which look familiar.

Sec 7 (p82) begins examining the non markovian case. Reiterates Equation 21, noting that in the non markovian case; Equation 22 is difficult to write s.t Equation 21 isn't necessarily solvable. It notes an additional step to try and reduce non markovian dynamics to markovian dynamics as:

$$\frac{d\rho(t)}{dt} = \int_0^t dt' k(t-t')\mathcal{L}(\rho(t')) \tag{23}$$

Which almost exactly corresponds to that introduced by (Strachan 2026), with $k(t-t')$ representing the memory effects w/$k(t-t') \to \delta(t-t')$ recovering markovian behaviour.

Note that (Strachan 2026) expresses this as:

$$\frac{d\rho(t)}{dt} = \int_0^t dt' k(t-t')\rho(t') \tag{24}$$

With the idea being inverting this eq to extract $k(t-t')$. Not entirely sure how to do that yet since the integral is over a different t' to $\rho(t)$…

Aside; but skimming over (Morillas-Rozas et al. 2026) seems to confirm my understanding acquired from the above reading; similar terms and logic poping up. Its not that papers main focus; but reassuring to note.

(Cerrillo and Cao 2014) likewise has similar terms. Both papers refer to "transfer tensors" seemingly as discreete objects which relate to the kernels somehow. Both papers seem to be exploring a "transfer tensor method" to extract certain system parameters including the memory kernel.

(Reimer et al. 2019) which compares multiple defs of quantum non markovianity (similarly to this project) has a section on memory kernels which echoes the same ideas as above.

Thinking ahead; in order to extract memory kernels; i mostly need to figure out how to invert Equation 25; the density ρ is known for all t; therefore it should just be a bit of calc; but unsure exactly how to do that given the different t' integration variable… not a today problem; will come back to it tmr…

11/10/2026

Right,

$$\frac{d\rho(t)}{dt} = \int_0^t dt' k(t-t')\rho(t') \tag{25}$$

Is an integral transform; that should provide a search term for figuring out how best to extract the kernel… Almost reminds me of greens functions too; although not sure how i'd set that up…

Did briefly wonder if i could simply U sub Equation 25 w/$u = t - t'$ then take $\frac{d}{du}$ derivative; however i doubt that would work

Skimming through a bunch of resources, it seems that the "integro-differential form" of Equation 25 is a major headache; hence motivating various methods which do not involve this convolution ("time convolutionless forms" according to (Rivas and Huelga 2012)). Thus far the only explicit mention of extracting the kernel i've seen is using transfer tensor methods (TTM) as in (Cerrillo and Cao 2014). So that is my next port of call.

Looking at some of my old notes; my understanding of greens functions are they are a way of dealing with inhomogeneous differential equations of the form:

$$\mathcal{D}f(x) = h(x) \tag{26}$$

Where $\mathcal{D}$ is a generalized differential operator. The green function $G(x-y)$ satisfies:

$$\mathcal{D}G(x-y) = \delta(x-y) \tag{27}$$

such that the solution for $f(x)$ follows:

$$f(x) = \int dy h(y) G(x-y) \tag{28}$$

If this understanding is true (would need to verify this), then Equation 25 could be expressed as a green function problem where $f(t) = \frac{d\rho(t)}{dt}$, $h(t') = \rho(t')$ and the kernel $G(t-t') = k(t-t')$. If this is the case, identifying $k(t-t')$ may be a matter of identifying $\mathcal{D}$, and solving Equation 27… Given that $\rho(t)$ evolves according to the dynamic map, I suspect $\mathcal{D}$ could be expressed in terms of the dynamic map. However there are a lot of unkowns with this potential approach, so will do some further reading first to double check i'm not missing a more straightforward approach.

Reading notes: (Cerrillo and Cao 2014):

Paper notes 2 approaches to quantum non markovianity:

1. Quantum master eqs such as the Nakajima-Zwansig formalism - these are exact but limited to certain limits (the same ones noted by (Rivas and Huelga 2012); weak coupling… or similar limits)
2. path integral simulations - adopts "harmonic bath assumption" which allows more flexibility but scales poorly w/bath size

(Cerrillo and Cao 2014) proposes a unified method which treats the underlying simulation as a black box which outputs system trajectories. The method can thus use these "approach agnostic trajectories" (encoded in the dynamical maps) to extract "Transfer tensors" which can then be used (among other things) to solve the Nakajima-Zwansiz Eq, and thus extract the original hamiltonian AND memory kernels.

The paper quickly outlines how to extract the dynamical map by considering trajectories from a full initial basis set for the system hilbert space (which is how this was implemented for this project).

The paper notes that for Markovian evolution, the dynamical maps relate to each other multiplicatively $\xi_n = \xi_1^n$, however this doesn't apply for non markovian evolution; thus requiring dynamic maps for every timestep to be found (which is inefficient, and currently what this project has implemented). The paper is thus interested in transfer tensors not only for the kernel extraction, but also to more efficiently extract the dynamic maps for non markovian evolution.

Note to self: (Cerrillo and Cao 2014) denotes the dynamical map as ξ_k for timesteps t_k s.t $\rho(t_k) = \xi_k \rho(0)$ - this is familiar just different notation

As such, the paper transforms the dynamical map to transfer tensors s.t transfer tensors can always propagate the evolution multiplicatively regardless of the non markovianity. It defines the following transformation:

$$T_{n,0} = \xi_n + \sum_{m=1}^{n-1} T_{n,m}\xi_m \tag{29}$$

for n timesteps. Therefore the time evolution follows as:

$$\rho(t_n) = \sum_{k=0}^{n-1} T_{n,k}\rho(t_k) \tag{30}$$

Ergo the transfer tensors represent some sense of temporal correlation between any 2 times in the evolution. Paper then notes simplifying assumptions (I.E, seperarable system-bath initial conditions, time independent hamiltonian) which result in time translational invariance in the transfer tensor s.t $T_{t,k} = T_{t-k} = T_{\Delta t}$. This allows for the transfer tensors to be extracted iteratively, starting w/$T_1 = \xi_1$:

- $T_2 = \xi_2 - T_1\xi_1$
- $T_3 = \xi_3 - T_1\xi_2 - T_2\xi_1$

Its a touch unclear how this procedure continues; presumably it follows from Equation 29 but am unsure…

The paper then explores the system propagation application of TTM.

It later discusses extracting the Nakajima-Zwansig eq and compares the transfer tensors to the terms in the somewhat more general form:

$$\frac{d\rho(t)}{dt} = -i\mathcal{L}\rho(t) + \int_0^t mathcalK(t-t')\rho(t')dt' \tag{31}$$

which relates to the transfer tensors as:

$$T_{k,n} = (1 - i\mathcal{L}\delta t)\delta_{k,n+1} + \mathcal{K}_{k,n}\delta t^2 \tag{32}$$

Where $\mathcal{L}$ is the system Liouvillian. Its unclear how to best extract the kernels; but the relationship is there…

Another paper (Morillas-Rozas et al. 2026) seems to discuss similar stuff and may give a more practical idea of how to do this. Actually skimming that immediately shows the relationship:

$$\lim_{\delta t \to 0}(\frac{T_k(\delta t)}{\delta t}) = \mathcal{K}(t)$$

Which… Given the finite nature of the simulation; may or may not work… Will have to see…

Reading notes: (Morillas-Rozas et al. 2026)

This paper notes the transfer tensor transform a touch differently:

$$T_k = \xi_k + \sum_{j=1}^{k} T_j \xi_{k-j} \tag{33}$$

w/the propagation following:

$$\rho(t_k) = \sum_{j=1}^{k} T_j \rho(t_{k-j}) \tag{34}$$

It also explicitly relates the Nakajima-Zwansig operators $\mathcal{P}, \mathcal{Q}$ to the dynamical map:

$$\xi_k = \mathcal{P}\mathcal{U}^k\mathcal{P} \tag{35}$$

where $\mathcal{U}(k\delta t) = \mathcal{U}^k$ is the time step propagator; so apparently the transfer tensor can be expressed as:

$$T_k = \mathcal{P}\mathcal{U}\mathcal{Q}(\mathcal{Q}\mathcal{U}\mathcal{Q})^{k-2}\mathcal{Q}\mathcal{U}\mathcal{P} \tag{36}$$

w/$\mathcal{Q} = 1 - \mathcal{P}$. It notes a slightly different Nakajima-Zwansig in terms of the dynamical map:

$$\frac{d\xi(t)}{dt} = \mathcal{P}\mathcal{L}\mathcal{P}\xi(t) + \int_0^t \mathcal{K}(t-t')\xi(t')dt' \tag{37}$$

W/the kernel being expressed in operator form as:

$$\mathcal{K}(t-t') = \mathcal{P}\mathcal{L}\mathcal{Q}exp(\mathcal{Q}\mathcal{L}\mathcal{Q}t)\mathcal{Q}\mathcal{L}\mathcal{P} \tag{38}$$

Yielding the aforementioned limit:

$$\lim_{\delta t \to 0}(\frac{T_k(\delta t)}{\delta t}) = \mathcal{K}(t) \tag{39}$$

The paper then goes on to compare the finite transfer tensors to the analytical kernel for a particular analytically solvable case study; w/fig 3 neatly demonstrating the convergence as $\delta t \to 0$. The paper explores the implications of these finite timestep differences…

Reading takeaways:

Given Equation 25, the most obviously tractable method is the Transfer Tensor method. There may be other methods in the literature, however this stands out as the most practicable. The questions that this entails are:

- How exactly does the iterative method work? Presumably Equation 29 yields this iteration, but am not 100% sure - need to look into it
- The transfer matrices converge onto the kernel for $\delta t \to 0$; however are distinct objects at the necessarily finite timesteps simulated by this project. (Morillas-Rozas et al. 2026) outlines this difference in a particular analytical case, however a general method for quantifying this error would be useful. If there are going to be finite time errors with this method, it would be useful to be able to quantify them.

The appendices of (Morillas-Rozas et al. 2026) may help; appendix A goes through the iteration explicitly; appendix B shows the limit Equation 39; however it goes over my head atm. Given appendix B, it may be possible to extract some error term, but its unclear. Ok exmaining appendix A, #eq-transferTensorTransform2 is a lot clearer and makes the iteration clearly explicit.

The sidenote has an idea for bounding the finite time step error. I'm unsure how rigorously true it is, but it may provide a start.

Cerrillo, Javier, and Jianshu Cao. 2014. "Non-Markovian Dynamical Maps: Numerical Processing of Open Quantum Trajectories." *Physical Review Letters* 112 (11): 110401. https://doi.org/10.1103/PhysRevLett.112.110401.

Morillas-Rozas, Marcel, Alberto López-García, Gonzalo Reina Rivero, Jianshu Cao, and Javier Cerrillo. 2026. "The Transfer Tensor Method: An Analytical Study Case." *The Journal of Chemical Physics* 165 (5): 054108. https://doi.org/10.1063/5.0340784.

Reimer, Viktor, Maarten Rolf Wegewijs, Konstantin Nestmann, and Mikhail Pletyukhov. 2019. "Five Approaches to Exact Open-System Dynamics: Complete Positivity, Divisibility and Time-Dependent Observables." *The Journal of Chemical Physics* 151 (4): 044101. https://doi.org/10.1063/1.5094412.

Rivas, Ángel, and Susana F Huelga. 2012. *Open Quantum Systems. An Introduction*. https://doi.org/https://doi.org/10.48550/arXiv.1104.5242.

Strachan, David J. 2026. "MarkovianityOutline."

A potentially crude manner of quantifying the finite timestep error; would be be considering the transfer tensor for timesteps $T(\delta t_1)$ and $T(\delta t_2)$ s.t $\delta t_1 > \delta t_2$. Given that $\frac{T}{\delta t^2}$ apparently converges in the $\delta t \to 0$ limit; presumably the error in $\frac{T(\delta t_1)}{\delta t_1^2}$ will be larger than in $\frac{T(\delta t_2)}{\delta t_2^2}$; so presumably $|\frac{T(\delta t_1)}{\delta t_1^2} - \frac{T(\delta t_2)}{\delta t_2^2}|$ relates to the error. This isn't necessarily an upper bound on that error, but may be a starting point… Actually… Cauchy convergence may provide an answer here. Examining my old analysis notes; if we let $a_n = \frac{T(\delta t_n)}{\delta t_n^2}$ s.t $\delta t_n > \delta t_{n+1}$, $lim_{n\to\infty}\delta t_n = 0$; the limit $\lim_{\delta t\to 0} \frac{T(\delta t)}{\delta t^2} = \lim_{n\to\infty} a_n = \alpha$ where α is the kernel limit. From the definition of cauchy convergence (that i have written down), it follows that $|a_n - \alpha| \leq |a_n - a_m| + |a_m - \alpha|$ for $m > n$. The $|a_n - a_m|$ term can easily be evaluated as above; the $|a_m - \alpha| < |a_n - \alpha|$. Therefore an upper bound for this error can be said to be $|a_n - \alpha| \leq |a_n - a_m| + |a_m - \alpha| \leq |a_n - a_m| + |a_n - a_m| = 2|a_n - a_m|$. This isn't an exact error, but it should bound the error from above for a particular δt_n I suspect that the smaller the difference $\delta t_n - \delta t_m$, the more closely this expression would bound the error. As such, assuming i've reasoned this correctly (and am using cauchy convergence appropriately), a bound on the error between the transfer tensor and the memory kernel would follow:

$$\epsilon(\delta t) \leq 2|\frac{T(\delta t + \nu)}{(\delta t + \nu)^2} - \frac{T(\delta t)}{\delta t^2}| \forall \nu \in [0, \infty]$$

where $\epsilon(\delta t) = |\frac{T(\delta t)}{\delta t^2} - \mathcal{K}|$ is the error between the transfer tensor and the memory kernel…